

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Бинарное дерево поиска.

Цель работы

Научиться работать с бинарными деревьями.

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. *Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. *Оценить сложность процедуры поиска по значению в бинарном дереве.

Описание метода решения задачи

Данная программа реализует бинарное дерево поиска (BST). Первая версия позволяет дубликаты, добавляя их в левое поддерево, вторая версия включает защиту от дубликатов через предварительный поиск перед вставкой. Основной метод решения основан на рекурсивных алгоритмах: вставка по принципу "меньшие - влево, большие - вправо", поиск за $O(h)$, где h - высота дерева. Подсчет вхождений требует полного обхода $O(n)$. Визуализация выполнена вертикальным выводом с отступами по уровням. Ключевое отличие между версиями - обработка дубликатов, влияющая на структуру дерева и производительность операций.

Анализ и выполнение заданий

4. Основные операции поиска и вставки в среднем случае $O(\log n)$ для сбалансированного дерева, в худшем случае $O(n)$ при вырождении в список. Ключевое отличие: вторая версия с проверкой дубликатов выполняет двойной обход при вставке поиск $O(h)$ + вставка $O(h)$, что дает $O(2h) = O(h)$, но с худшей константой. Подсчет вхождений в обеих версиях $O(n)$ так как требует полного обхода.

Листинг

Файл LR4.cpp (1 задание)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

//Структура узла бинарного дерева
struct Node {
    int data; //Значение узла
    struct Node* left; //Указатель на левое поддерево
```

```

    struct Node* right;//Указатель на правое поддерево
};
struct Node* root = NULL;//Корень дерева

//Функция проверки, является ли строка числом
int is_number(const char* str) {
    //Проверка на пустую строку или NULL
    if (str == NULL || *str == '\0') return 0;

    //Проверяем каждый символ строки
    for (int i = 0; str[i] != '\0'; i++) {
        // Разрешаем цифры и минус только в начале числа
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0;//Не число
        }
    }
    return 1;//Строка является числом
}

//Функция создания дерева (рекурсивная)
struct Node* CreateTree(struct Node* r, int data)
{
    //Если достигли пустого узла - создаем новый
    if (r == NULL)
    {
        // Выделение памяти для нового узла
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL)
        {
            printf("Ошибка выделения памяти");
            exit(0);
        }
        //Инициализация полей нового узла
        r->left = NULL;
        r->right = NULL;
        r->data = data;
        return r;
    }
    //Рекурсивная вставка в правое поддерево если данные больше
    //текущего узла
    if (data > r->data)
        r->right = CreateTree(r->right, data);
    //Рекурсивная вставка в левое поддерево если данные меньше или
    //равны
    else
        r->left = CreateTree(r->left, data);
    return r;
}

//Функция вывода дерева в консоль (вертикальный вывод)
void print_tree(struct Node* r, int l)
{
    //Пустое дерево
    if (r == NULL)
    {
        return;
    }

```

```

    }
    //Рекурсивный вывод правого поддерева
    print_tree(r->right, l + 1);
    //Вывод отступов для визуализации уровня узла
    for (int i = 0; i < l; i++)
    {
        printf("  ");
    }
    //Вывод данных текущего узла
    printf("%d\n", r->data);
    //Рекурсивный вывод левого поддерева
    print_tree(r->left, l + 1);
}

//Функция поиска элемента в дереве (рекурсивная)
struct Node* search(struct Node* r, int key)
{
    //Пустое дерево или найденный элемент
    if (r == NULL || r->data == key)
        return r;
    //Рекурсивный поиск в правом поддереве если ключ больше
    if (key > r->data)
        return search(r->right, key);
    //Рекурсивный поиск в левом поддереве если ключ меньше
    else
        return search(r->left, key);
}

//Функция подсчёта числа вхождений элемента в дерево
int count_occurrences(struct Node* r, int key)
{
    //Пустое дерево
    if (r == NULL)
        return 0;
    int count = 0;
    //Если нашли совпадение - увеличиваем счетчик
    if (r->data == key)
        count = 1;
    //Рекурсивный подсчет в левом и правом поддеревьях
    return count + count_occurrences(r->left, key) +
count_occurrences(r->right, key);
}

int main()
{
    setlocale(LC_ALL, "");
    char input[100]; // Буфер для ввода пользователя
    int D; // Числовое значение введенных данных
    root = NULL; // Инициализация корня дерева
    printf("Вводите числа для построения дерева.\n");
    printf("Для окончания ввода введите 'stop'\n\n");

    //Построение дерева
    while (1)
    {
        printf("Введите число: ");
    }

```

```

//Чтение ввода пользователя
if (scanf("%99s", input) != 1) {
    printf("Ошибка ввода.\n");
    continue;
}

//Проверка на команду остановки ввода
if (strcmp(input, "stop") == 0) {
    printf("Построение дерева окончено\n\n");
    break;
}

//Проверка, является ли ввод числом
if (!is_number(input)) {
    printf("Ошибка: введите число или 'stop' для
завершения.\n");
    continue;
}

//Преобразование строки в целое число
D = atoi(input);
//Добавление элемента в дерево
root = CreateTree(root, D);
}

//Вывод построенного дерева
printf("Построенное дерево:\n");
print_tree(root, 0); // Начальный уровень = 0
printf("\n");

//Поиск элементов в дереве
while (1) {
    printf("Введите значение для поиска (или 'stop' для
завершения): ");
    if (scanf("%99s", input) != 1) {
        printf("Ошибка ввода.\n");
        continue;
    }

    //Проверка на команду остановки поиска
    if (strcmp(input, "stop") == 0) {
        break;
    }

    //Проверка корректности ввода
    if (!is_number(input)) {
        printf("Ошибка: введите число или 'stop' для
завершения.\n");
        continue;
    }

    int search_key = atoi(input); // Ключ для поиска
    struct Node* result = search(root, search_key); // Поиск в
дереве

    //Вывод результата поиска

```

```

        if (result != NULL)
            printf("Элемент %d найден в дереве.\n", search_key);
        else
            printf("Элемент %d не найден в дереве.\n", search_key);
    }

    //Подсчет вхождений элементов
    while (1) {
        printf("Введите значение для подсчета вхождений (или 'stop'
для завершения): ");
        if (scanf("%99s", input) != 1) {
            printf("Ошибка ввода.\n");
            continue;
        }

        //Проверка на команду остановки подсчета
        if (strcmp(input, "stop") == 0) {
            break;
        }

        //Проверка корректности ввода
        if (!is_number(input)) {
            printf("Ошибка: введите число или 'stop' для
завершения.\n");
            continue;
        }

        int count_key = atoi(input); //Ключ для подсчета
        //Подсчет количества вхождений элемента
        int occurrences = count_occurrences(root, count_key);
        printf("Элемент %d встречается в дереве %d раз(а).\n",
count_key, occurrences);
    }
    printf("Программа завершена. Нажмите Enter для выхода...");
    getchar();
    getchar();
    return 0;
}

```

Файл 4LR_star.cpp (2 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

//Структура узла бинарного дерева поиска
struct Node {
    int data; // Значение узла
    struct Node* left; // Указатель на левое поддерево (меньшие
значения)
    struct Node* right; // Указатель на правое поддерево (большие
значения)
};

struct Node* root = NULL; // Корень дерева

```

```

//Функция проверки, является ли строка числом
int is_number(const char* str) {
    //Проверка на пустую строку или NULL указатель
    if (str == NULL || *str == '\0') return 0;
    //Посимвольная проверка строки
    for (int i = 0; str[i] != '\0'; i++) {
        //Разрешаем цифры и минус
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0; // Обнаружен недопустимый символ
        }
    }
    return 1; // Строка содержит корректное число
}

//Рекурсивная функция поиска элемента в дереве
struct Node* search(struct Node* r, int key) {
    //Пустой узел или найденное значение
    if (r == NULL || r->data == key)
        return r;
    //Рекурсивный поиск в правом поддереве для больших значений
    if (key > r->data)
        return search(r->right, key);
    //Рекурсивный поиск в левом поддереве для меньших значений
    else
        return search(r->left, key);
}

//Функция создания дерева с защитой от дубликатов
struct Node* CreateTree(struct Node* r, int data) {
    //Создание нового узла при достижении пустой позиции
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Ошибка выделения памяти");
            exit(0);
        }
        //Инициализация полей нового узла
        r->data = data;
        r->left = NULL;
        r->right = NULL;
        return r;
    }
    //Вставка в правое поддерево для значений больше текущего узла
    if (data > r->data) {
        r->right = CreateTree(r->right, data);
    }
    //Вставка в левое поддерево для значений меньше текущего узла
    else if (data < r->data) {
        r->left = CreateTree(r->left, data);
    }
    //Если значение равно текущему узлу - дубликат игнорируется
    return r;
}

//Функция визуального вывода дерева

```

```

void print_tree(struct Node* r, int l) {
    //Пустое поддерево
    if (r == NULL) {
        return;
    }
    //Рекурсивный вывод правого поддерева
    print_tree(r->right, l + 1);
    //Формирование отступов
    for (int i = 0; i < l; i++) {
        printf("  ");
    }
    //Вывод значения текущего узла
    printf("%d\n", r->data);
    //Рекурсивный вывод левого поддерева
    print_tree(r->left, l + 1);
}

//Функция подсчета количества вхождений элемента в дерево
int count_occurrences(struct Node* r, int key) {
    //Пустое поддерево
    if (r == NULL)
        return 0;

    int count = 0;
    //Увеличение счетчика при совпадении значения
    if (r->data == key)
        count = 1;

    //Рекурсивный подсчет в левом и правом поддеревьях
    return count + count_occurrences(r->left, key) +
count_occurrences(r->right, key);
}

int main() {
    setlocale(LC_ALL, "");
    char input[100]; // Буфер для ввода пользователя
    int D; // Для введенных данных
    root = NULL; // Инициализация пустого дерева
    printf("Вводите числа для построения дерева.\n");
    printf("Для окончания ввода введите stop\n\n");

    //Построение бинарного дерева поиска
    while (1) {
        printf("Введите число: ");
        //Чтение ввода пользователя с ограничением длины
        if (scanf("%99s", input) != 1) {
            printf("Ошибка ввода.\n");
            continue;
        }

        //Проверка команды завершения ввода
        if (strcmp(input, "stop") == 0) {
            printf("Построение дерева окончено\n\n");
            break;
        }
    }
}

```

```

//Проверка что введено число
if (!is_number(input)) {
    printf("Ошибка: введите число или stop для
завершения.\n");
    continue;
}

//Преобразование строки в целое число
D = atoi(input);
//Проверка на дубликат перед добавлением
if (search(root, D) != NULL) {
    printf("Ошибка: число %d уже существует в дереве.
Введите другое число.\n", D);
    continue;
}
//Добавление уникального элемента в дерево
root = CreateTree(root, D);
}
printf("Построенное дерево:\n");
print_tree(root, 0); //Вывод начиная с нулевого уровня
printf("\n");

//Поиск элементов
while (1) {
    printf("Введите значение для поиска (или 'stop' для
завершения): ");
    if (scanf("%99s", input) != 1) {
        printf("Ошибка ввода.\n");
        continue;
    }

    //Проверка команды завершения поиска
    if (strcmp(input, "stop") == 0) {
        break;
    }

    //Валидация ввода для поиска
    if (!is_number(input)) {
        printf("Ошибка: введите число или 'stop' для
завершения.\n");
        continue;
    }

    int search_key = atoi(input); //Ключ для поиска
    //Выполнение поиска в дереве
    struct Node* result = search(root, search_key);
    //Вывод результата поиска
    if (result != NULL)
        printf("Элемент %d найден в дереве.\n", search_key);
    else
        printf("Элемент %d не найден в дереве.\n", search_key);
}

//Подсчет количества вхождений элементов
while (1) {

```

```

        printf("Введите значение для подсчета вхождений (или 'stop'
для завершения): ");
        if (scanf("%99s", input) != 1) {
            printf("Ошибка ввода.\n");
            continue;
        }

        //Проверка команды завершения подсчета
        if (strcmp(input, "stop") == 0) {
            break;
        }

        //Валидация ввода для подсчета
        if (!is_number(input)) {
            printf("Ошибка: введите число или 'stop' для
завершения.\n");
            continue;
        }

        int count_key = atoi(input); //Ключ для подсчета
        //Подсчет количества вхождений элемента
        int occurrences = count_occurrences(root, count_key);
        printf("Элемент %d встречается в дереве %d раз(a).\n",
count_key, occurrences);
    }
    printf("Программа завершена. Нажмите Enter для выхода...");
    getchar();
    getchar();
    return 0;
}

```

Текст к программе

1. 4LR.cpp (1 задание)

Реализует классическое бинарное дерево поиска, где вставка элементов происходит без проверки на дубликаты. При добавлении нового значения алгоритм рекурсивно сравнивает его с текущими узлами: если значение больше - идет в правое поддерево, если меньше или равно - в левое. Это позволяет дубликатам накапливаться в левых поддеревьях. Поиск работает по тому же принципу: находит первый подходящий элемент, но может не обнаружить все дубликаты. Подсчет вхождений требует полного рекурсивного обхода всего дерева, так как дубликаты могут быть разбросаны по разным ветвям. Визуализация показывает реальную структуру с возможными повторениями значений.

2. LR4_star.c (2 задание)

Модифицированная версия с полной проверкой уникальности элементов. Перед каждой вставкой выполняется рекурсивный поиск по всему дереву. Если элемент уже существует, вставка блокируется с выводом сообщения об ошибке. Сама функция вставки также изменена: строгое сравнение (больше/меньше) без обработки равенств. Это гарантирует, что в дереве не будет повторяющихся значений. Поиск всегда возвращает однозначный результат, а подсчет

вхождений фактически работает как проверка наличия элемента (возвращает 0 или 1). Визуализация отображает чистое дерево без дубликатов.

Результаты работы программы

Результаты работы программы показаны на рисунках 1-12.

I. 4LR.cpp (1 задание)

```
Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     7
    6
   5
  4
 3
```

Рисунок 1 – Создание дерева и его вывод

```
Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     7
    6
   5
  4
 3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
```

Рисунок 2 – Число найдено

```

Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     /
    7
   /
  6
 /
5
/
4
/
3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
Введите значение для поиска (или stop для завершения): 1
Элемент 1 не найден в дереве.
Введите значение для поиска (или stop для завершения):

```

Рисунок 3 – Число не было найдено

```

Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     /
    7
   /
  6
 /
5
/
4
/
3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
Введите значение для поиска (или stop для завершения): 1
Элемент 1 не найден в дереве.
Введите значение для поиска (или stop для завершения): f
Ошибка: введите число или stop для завершения.

```

Рисунок 4 – Попытка поиска буквы

```

Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     /
    7
   /
  6
 /
5
/
4
/
3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
Введите значение для поиска (или stop для завершения): 1
Элемент 1 не найден в дереве.
Введите значение для поиска (или stop для завершения): f
Ошибка: введите число или stop для завершения.
Введите значение для поиска (или stop для завершения): stop
Введите значение для подсчета вхождений (или stop для завершения): 4
Элемент 4 встречается в дереве 1 раз(a).

```

Рисунок 5 – Успешный подсчет количества вхождений

```

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     /
    7
   /
  6
 /
5
/
4
/
3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
Введите значение для поиска (или stop для завершения): 1
Элемент 1 не найден в дереве.
Введите значение для поиска (или stop для завершения): f
Ошибка: введите число или stop для завершения.
Введите значение для поиска (или stop для завершения): stop
Введите значение для подсчета вхождений (или stop для завершения): 4
Элемент 4 встречается в дереве 1 раз(a).
Введите значение для подсчета вхождений (или stop для завершения): f
Ошибка: введите число или stop для завершения.

```

Рисунок 6 – Попытка подсчитать количество вхождений буквы

```

Введите число: 4
Введите число: 5
Введите число: 6
Введите число: 7
Введите число: 9
Введите число: 3
Введите число: stop
Построение дерева окончено

Построенное дерево:
      9
     /
    7
   /
  6
 /
5
/
4
/
3

Введите значение для поиска (или stop для завершения): 9
Элемент 9 найден в дереве.
Введите значение для поиска (или stop для завершения): 1
Элемент 1 не найден в дереве.
Введите значение для поиска (или stop для завершения): f
Ошибка: введите число или stop для завершения.
Введите значение для поиска (или stop для завершения): stop
Введите значение для подсчета вхождений (или stop для завершения): 4
Элемент 4 встречается в дереве 1 раз(a).
Введите значение для подсчета вхождений (или stop для завершения): f
Ошибка: введите число или stop для завершения.
Введите значение для подсчета вхождений (или stop для завершения): stop
Программа завершена. Нажмите Enter для выхода...

C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\LR4\Debug\LR4.exe (процесс 38892) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при оста
новке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:

```

Рисунок 7 – Корректное завершение программы

II. LR4_star.cpp (2 задание)

```

Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: f
Ошибка: введите число или stop для завершения.
Введите число:

```

Рисунок 8 – Попытка добавления буквы

```

Вводите числа для построения дерева.
Для окончания ввода введите stop

Введите число: f
Ошибка: введите число или stop для завершения.
Введите число: 5
Введите число: 5
Ошибка: число 5 уже существует в дереве. Введите другое число.
Введите число:

```

Рисунок 9 – Попытка добавления дубликата

Выводы

Оба варианта реализации бинарного дерева поиска демонстрируют классические алгоритмы работы с BST. Первая версия показывает базовый принцип работы с простой вставкой, но допускает дубликаты, что не соответствует строгому определению BST. Вторая версия исправляет этот недостаток за счет двойного обхода при вставке, обеспечивая уникальность элементов.